How RAG QA Lab works (a guide for students)

1. The problem, and the idea behind RAG

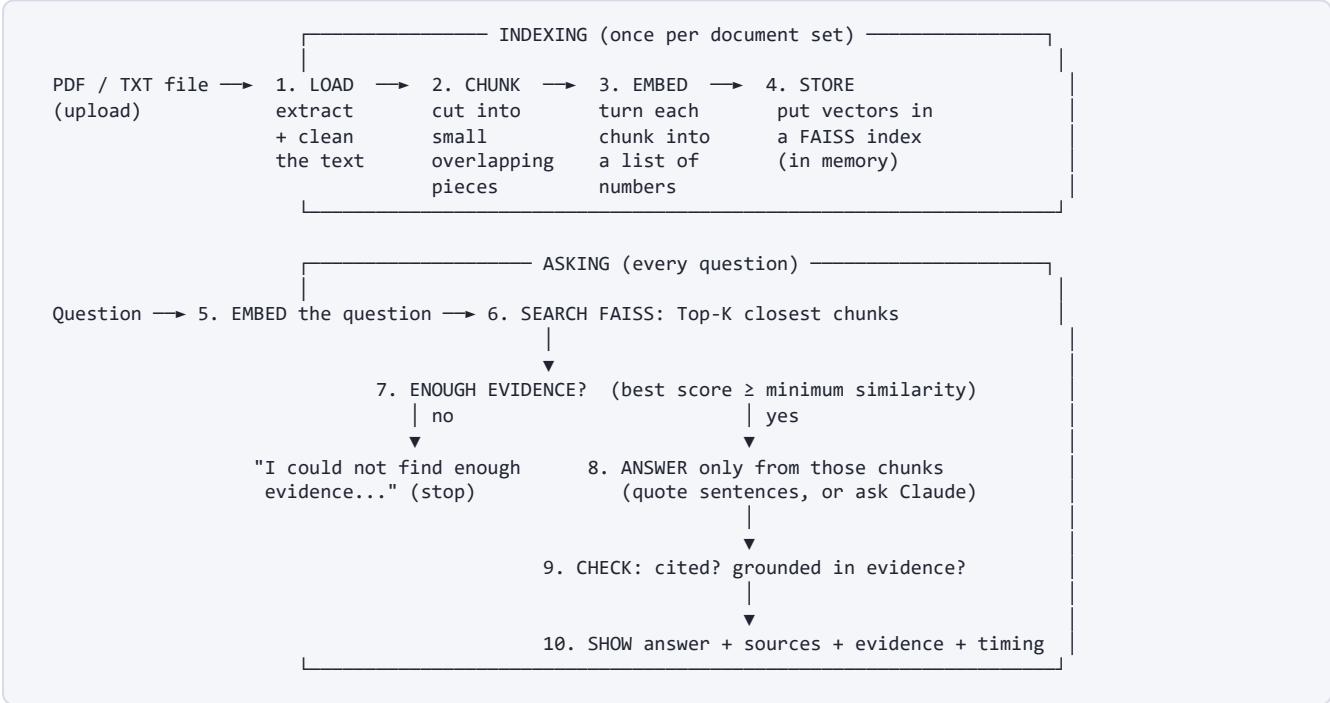
A language model such as ChatGPT or Claude only knows what it learned in training. It has never read **your** company policy PDF, and if you ask about it, it may confidently make something up (a *hallucination*).

RAG = Retrieval-Augmented Generation. Instead of asking the model to remember, we let it *look things up first*:

> Think of an **open-book exam**. The student (the answer step) is not allowed to guess from memory. A **librarian** (the retrieval step) first finds the 3-4 most relevant pages in the book, and the answer may only use those pages. If the librarian finds nothing relevant, the correct answer is "I can't find it in the book."

This project is that idea, plus a **QA layer** that measures whether it really behaves that way.

2. The whole system on one page



(Section 3 lists the technology behind each step.) Code map: steps 1 `rag/loaders.py` · 2 `rag/chunking.py` · 3 `rag/embeddings.py` · 4 `rag/vector_store.py` · 5-7 `rag/retriever.py` · 8 `rag/answering.py` (or `llm/claude_provider.py`) · 9 `qa/deterministic.py` · UI `app.py`.

3. Technologies used

Everything runs **locally on your computer** in one Python program. The only things that touch the internet are the one-time download of the embedding model and, if you choose Claude mode, the Claude API call.

Layer	Technology	What it does in this project	Where
Language	**Python** (3.10+, built on 3.14)	Everything is plain Python.	all `.py` files
Web interface	**Streamlit**	Turns a Python script into an interactive web page: sidebar, buttons, tabs, tables, charts. No HTML/JavaScript needed.	`app.py`
PDF reading	**PyMuPDF** (`pymupdf`)	Fast PDF text extraction, page by page.	`rag/loaders.py`
Meaning → numbers	**Sentence Transformers** with the model `all-MiniLM-L6-v2`	Converts text into 384-number vectors (embeddings). The model is small (about 90 MB) and runs on an ordinary CPU.	`rag/embeddings.py`
Deep-learning engine	**PyTorch** and Hugging Face **Transformers**	The libraries Sentence Transformers runs on (installed automatically).	(dependency)
Vector search	**FAISS** (`faiss-cpu`, by Meta AI)	Stores the vectors and finds the closest ones to a question in milliseconds. We use an *exact* index (`IndexFlatIP`, inner product = cosine similarity on normalised vectors).	`rag/vector_store.py`
Number crunching	**NumPy**	Holds the vectors as arrays and does the maths.	`rag/embeddings.py`
Tables	**pandas**	Builds the results table on the QA dashboard.	`app.py`
Optional answer writer	**Claude API** (Anthropic Python SDK `anthropic`)	In Claude mode, writes the answer from the retrieved evidence only. Kept in its own folder so the retrieval code never depends on it.	`llm/claude_provider.py`
Optional evaluation library	**RAGAS**	An open-source toolkit for scoring RAG systems (faithfulness, context precision/recall, answer relevancy) using a judge model. Plugged in through two small adapters so it uses *our* Claude client and *our* embedding model.	`qa/ragas_eval.py`
Settings and secrets	**python-dotenv** and a `.env` file	Reads settings and the API key from the environment; the key is never written in code and `.env` is git-ignored.	`rag/config.py`
Automated testing	**pytest**	110 tests: chunking, retrieval, source attribution, empty documents, irrelevant questions, unsupported answers, the evaluation runner.	`tests/`
Packaging	**venv** and **pip**	An isolated environment (`.venv`) and the dependency list `requirements.txt`.	project root

Only for making this guide: the **markdown** package turned this file into HTML and Microsoft Edge printed it to PDF.

How Streamlit works (it explains several behaviours you will see)

1. **Every click re-runs the whole script from top to bottom.** When you press *Ask*, Streamlit runs `app.py` again from the first line and redraws the page.
2. So that data survives those re-runs, the app keeps it in `st.session_state`: the uploaded documents, the search index, and the last answer. Session state belongs to **one browser tab**, which is why *refreshing the page loses the index*.
3. Loading the embedding model takes seconds, so it is kept in `st.cache_resource`: loaded once per server and shared. That is why the *first* indexing after starting the server is slow and later ones are fast.
4. The page is built from Streamlit **widgets**: `st.sidebar` (left panel), `st.file_uploader`, `st.slider`, `st.number_input`, `st.checkbox`, `st.radio`, `st.tabs` (Ask / QA dashboard), `st.form` (question box + Ask button), `st.metric` (the big numbers), `st.expander` (evidence rows), `st.dataframe` and `st.bar_chart` (dashboard).
5. `.streamlit/config.toml` holds Streamlit's own settings (for example, the source file watcher is turned off).

How the pieces are organised (separation of concerns)

```
app.py ........... the screen (Streamlit)              ← what the user sees
rag/ ............. the RAG engine: load, chunk, embed, store, retrieve, answer   ← no LLM inside
llm/ ............. optional Claude connection           ← plugs in from outside
qa/ .............. measuring quality: rule-based checks, optional LLM judge, runner
tests/ ........... pytest, proves each part works
data/ ............ sample policies + the evaluation questions
```

Design rule: **the retrieval engine never imports the LLM code** (a test enforces this). That is why the app works with no API key, and why you can swap Claude for another model without touching retrieval.

4. What happens when you upload a large PDF (step by step)

Imagine a 100-page PDF.

Step 1 - Load. PyMuPDF reads every page and extracts the text. PDFs store text in hard-wrapped lines, so we glue broken lines back into sentences and tidy up spaces. We remember where each page starts so we can later say "this came from page 7". *If the PDF is a scanned image with no real text, nothing can be extracted (there is no OCR), and the app tells you the file was skipped.*

Step 2 - Chunk. We cannot compare a question with 100 pages at once, so we cut the text into **chunks** of about 800 characters (roughly a paragraph). Each chunk **overlaps** the previous one by about 120 characters so a sentence that falls on a boundary is not lost. We prefer to cut at paragraph or sentence ends. Every chunk gets an ID such as `policy.pdf::chunk-0042` and a page number. *100 pages ≈ 300,000 characters ≈ 450 chunks.*

Step 3 - Embed. An *embedding model* (`all-MiniLM-L6-v2`) turns each chunk into a list of 384 numbers, a **vector**, that captures its *meaning*. Texts with similar meaning get vectors that point in similar directions. "How long can I work from home?" and "employees may work remotely up to 3 days" end up close

together even though they share few words.

Step 4 - Store. All vectors go into **FAISS**, a library for fast similarity search. Our index is *exact* (it compares against every chunk); for hundreds or thousands of chunks that takes a few milliseconds. *The index lives in memory: 450 chunks ≈ 0.7 MB. It is rebuilt each time you index, and it is lost if you refresh the page.*

Step 5-6 - Ask & search. Your question is embedded with the same model, and FAISS returns the **Top-K** chunks whose vectors are closest. Closeness is a **cosine similarity score**: about 1.0 means "same meaning", about 0 means "unrelated".

Step 7 - Enough evidence? If even the best chunk scores below the **minimum similarity**, the app refuses with an explicit message. It never calls the answer step, so it cannot invent anything.

Step 8 - Answer, using only the evidence. - *Retrieval-only mode (no API key):* every sentence of the retrieved chunks is scored against the question. The best few (similarity ≥ 0.50) are **quoted word for word** with their source. Nothing is generated, so nothing can be made up, but it cannot summarise or combine facts. - *Claude mode (optional):* the evidence and question are sent to Claude with strict rules: "use only this evidence, cite it, or reply INSUFFICIENT_EVIDENCE".

Step 9 - Check. Two rule-based checks run on every answer: **source attribution** (every cited chunk was really retrieved) and **grounding** (the answer's words and numbers appear in the evidence; an invented "20 characters" would be flagged).

Step 10 - Show. The answer, the sources, the evidence text and the timings.

Worked example (the bundled sample documents)

Question: "*How many days per week can employees work remotely?*" → best chunk `remote_work_policy.txt::chunk-0001`, score **0.764** (≥ 0.30, so enough evidence) → answer: "*Eligible employees may work remotely up to 3 days per week. [remote_work_policy.txt · chunk-0001]*" Question: "*What is the capital of France?*" → best score ≈ **0.01**, far below 0.30 → "*I could not find enough evidence…*"

5. Every item on the website

Sidebar - 1 · Documents

Item	What it does
Upload box	Choose PDF/TXT files (200 MB each). Nothing is indexed yet.
Index uploads	Runs steps 1-4 on the chosen files and builds the searchable index.
Load samples	Indexes the 4 bundled policy files (3 TXT + 1 PDF) so you can try the app instantly.
File dialog not opening? Index by path	Alternative to the upload box: type a file/folder path on this computer.
Clear all documents	Empties the document list and the index.

Sidebar - 2 · Retrieval settings

Setting	Meaning	If too small	If too large
Chunk size (characters), default 800	How long each piece of text is.	Chunks lose context ("it", "the limit" - which limit?).	Chunks mix several topics, scores get blurry, more text to read.
Chunk overlap (characters), default 120	How many characters two neighbouring chunks share. Must be smaller than the chunk size.	A fact cut in half at a boundary can be missed.	Lots of duplicated text, bigger index, repeated sentences in results.
Top-K chunks, default 4	How many best-matching chunks are fetched as evidence.	The answer may be in chunk #5 and is never seen.	Weak, irrelevant chunks creep in and add noise (and cost, with an LLM).
Embedding backend	Which model makes the vectors. `auto` = real model if it loads, else the fallback. `sentence-transformers` = real model only. `hashing` = offline fallback.		
Set evidence threshold manually → Minimum similarity	The refusal gate (step 7). Unticked, the app uses the model's recommended value (0.30).	Too low: it answers questions it should refuse.	Too high: it refuses questions it could answer.
Latency budget (ms) for QA	Only used by the QA dashboard: a case fails its speed check if it takes longer than this.		

Rule of thumb: whenever you change chunk size, overlap, backend or threshold, the index must be rebuilt. The app warns you with "Settings changed since the index was built" and a Rebuild button.

The hashing fallback is not "smart". It only matches shared words, so "password length" will not match "at least 14 characters". The app warns you in yellow when it is active.

Sidebar - 3 · Answer mode

Option	Meaning
Retrieval-only (no API key needed)	Quotes sentences from the evidence. Free, offline, cannot hallucinate.
Claude (optional)	A language model writes the answer from the evidence only. Needs `ANTHROPIC_API_KEY` (or paste a key into the box, kept only for the session). Costs API credits.

Main page - top status strip

Documents indexed, **Chunks** (how many pieces were made), **Index build** (seconds to chunk + embed), **Min similarity** (the active refusal threshold), and **Embedder** (which model is really in use).

Ask tab

- **Question box + Ask** - type a question; steps 5-10 run.
- **Green box** = answer with citations like `[remote_work_policy.txt · chunk-0001]`. **Yellow box** = refusal ("not enough evidence") with a *Why:* line that shows the score vs. the threshold.
- **Retrieval / Generation / Total** - milliseconds spent searching, producing the answer, and overall. **Top score** - similarity of the best chunk. **Answer mode** - `extractive`, `llm` or `none` (refused).
- **Answer checks (deterministic, no LLM)** - ✅/❌ for source attribution and grounding.
- **Retrieved evidence (Top-K)** - one expandable row per chunk: rank, file, chunk number, page (PDFs), similarity score, "✓ cited" if the answer used it, plus the exact text and character range. **This is the proof** - students can always check the answer against the source.

QA dashboard tab - "How good is the system?"

It runs 17 prepared questions (13 answerable, 4 not answerable) from `data/eval_dataset.json` and measures the real behaviour. | Item | Meaning | |---|---| | **Test questions** (radio) | **Bundled sample questions** = the fixed 17 questions about the sample policy files. **My own questions** = a table where you write questions (with a reference answer) about the documents *you* uploaded; Claude can also draft some for you to review. The evaluation, and RAGAS, always judge exactly these questions, so to evaluate your own PDF you must choose this option. | | **Pass rate (deterministic)** | Share of cases where *all* rule-based checks passed. | | **Evidence coverage** | For answerable questions: share of the "must-have" facts that actually appear in the retrieved chunks. Measures **retrieval** quality. | | **Correct abstention** | For unanswerable questions: share where the app correctly said "not enough evidence". Measures **honesty**. | | **Mean / p95 latency** | Average time, and the time within which 95% of questions finished. | | **Results table + bar chart** | PASS / FAIL / SKIPPED per case, failed checks, scores, timing. | | **Case details** | For each case the answer and every check with its reason. | | **RAGAS metrics section** | Optional, at the bottom (needs RAGAS installed and Claude mode). Four judge-model scores per answered case: **Faithfulness** (are the answer's claims backed by the evidence?), **Context precision** (are the useful chunks at the top of the list?), **Context recall** (does the evidence contain everything the reference answer says?), **Answer relevancy** (does the answer address the question?). Costs API credits and is non-deterministic. | | **LLM judge checkbox** | Optional (Claude mode only): asks a model to rate faithfulness. **Non-deterministic** (can vary or be wrong), so it never changes PASS/FAIL. | | **SKIPPED** | A case that needs a real semantic model while only the word-matching fallback is active. It is *not* counted as a pass. |

Deterministic vs LLM-based evaluation: a *deterministic* check is a fixed rule (does the answer contain "14 characters"? was the cited chunk retrieved?). Same input always gives the same result, and it is free. An *LLM-based* check is a model's opinion: flexible, but it can differ run to run and make mistakes. This project keeps them strictly separate.

Where RAGAS fits. RAGAS is a well-known open-source library that packages several LLM-based RAG metrics. We use it as an *extra opinion* next to our own rule-based checks, never instead of them. Think of the rule-based checks as an automatic marking scheme (exact, repeatable) and RAGAS as a second examiner who reads the answer and gives a score (more flexible, but can disagree with itself on another day). To score *context precision* and *context recall*, RAGAS needs the "model answer", so every answerable test case in `data/eval_dataset.json` has a one-sentence `reference`.

6. Honest limitations (good teaching points)

1. **Scanned PDFs** are not supported (no OCR).
2. The index is **in memory**: refreshing the page loses it, and every change re-embeds all documents.
3. Retrieval-only answers are **quotes**, not summaries; they may include a loosely related third sentence.
4. Thresholds (0.30 for chunks, 0.50 for sentences) were chosen from only **17 test cases**. 100% on this dataset does not mean 100% on your documents - which is exactly why the dashboard exists and why you should add your own test questions.
5. The grounding check is **lexical**: a good paraphrase may be flagged, and a distorted answer using the same words may pass.

7. Classroom exercises

1. Load samples. Ask about a fact, then something unrelated. Compare the green and yellow outcomes and their scores.
2. Set **Chunk size** to 200, rebuild, and ask the same question. Does the evidence still contain the answer?
3. Set **Top-K** to 1 versus 8 and watch the evidence list and the answer change.
4. Tick **Set evidence threshold manually**, push it to 0.8, and see correct questions get refused. Then push it to 0.05 and see unrelated questions get answered.
5. Switch **Embedding backend** to `hashing`, ask "What is the minimum password length?" and explain why it fails (vocabulary mismatch), then switch back.
6. Upload your own PDF, write 5 questions (2 unanswerable), and add them to `data/eval_dataset.json`.